

PHASE 2: Data Processing

***Goal:** Extract content from source files, filter out noise, and apply a chunking strategy to prepare unstructured data for embedding and retrieval.*

1. Data Extraction

Core Concept

Data extraction is the process of parsing raw unstructured files (e.g., PDFs, HTML, Word docs) into structured text. Because LLMs only process text, binary files must be decoded and their meaningful text extracted accurately [1].

How it Works on Databricks

Databricks offers two main approaches to parsing document content:

- **Python Packages (via PySpark UDFs):** For custom extraction, you can wrap Python libraries like `unstructured`, `PyPDF2`, or `PyMuPDF` in a Spark User-Defined Function (UDF). This allows you to parallelize the parsing of millions of documents across a Databricks cluster [1].
- **Built-in AI Parsing:** Databricks also offers the native `ai_parse_document()` function (in Databricks Runtime 17.3+) which leverages Foundation Model APIs to automatically parse PDFs and images, extracting text and even generating descriptions for figures [2].

When to Use vs. Alternatives

Use Python packages like `unstructured` when you need granular control over the parsing logic (e.g., preserving specific HTML tags or targeting specific metadata). Use the native `ai_parse_document` function for rapid, managed parsing of complex PDFs containing figures and tables without managing external Python dependencies [2].

Common Mistakes

A frequent mistake is failing to extract metadata during the parsing phase. The Databricks AI Cookbook notes that extracting metadata (like document source, headings, and structure) at this stage is "essential for optimal performance" as it helps narrow down retrieved information later [1].

```
import pandas as pd
from pyspark.sql.functions import pandas_udf
from unstructured.partition.pdf import partition_pdf

@pandas_udf("string")
def extract_text_unstructured(file_bytes_series: pd.Series) -> pd.Series:
    def parse_doc(file_bytes):
        import io
        # Parse PDF bytes using the unstructured package
        elements = partition_pdf(file=io.BytesIO(file_bytes))
        return "\n".join([str(el) for el in elements])
    return file_bytes_series.apply(parse_doc)

# Applying the UDF to our raw document dataframe
parsed_df = df.withColumn("parsed_text", extract_text_unstructured("content"))
```

Function Explanation: @pandas_udf registers a vectorized Python function that takes a batch of raw binary file bytes (from our Auto Loader phase) and applies the unstructured package to extract the textual elements. This allows the heavy Python parsing logic to run distributed across Databricks worker nodes.

2. Data Cleaning & Filtering

Core Concept

Before chunking, the parsed text must be enriched and filtered. Filtering removes extraneous content that degrades RAG quality, while deduplication ensures the index isn't bloated with repetitive information [1].

How it Works on Databricks

- **Filtering:** Eliminating irrelevant boilerplate (e.g., navigation menus from scraped HTML, or repetitive headers/footers in PDFs).
- **Deduplication:** Using techniques like MinHash to identify and drop near-duplicate documents via Spark similarity joins [1].

3. Chunking Strategies

Core Concept

Chunking breaks parsed text into smaller units so it fits within the LLM's context window and minimizes distracting information. Databricks highlights that the choice of chunking strategy is "one of the first layers of optimization in an RAG application" [1].

How it Works on Databricks (Advanced Strategies)

Databricks outlines several chunking strategies you can apply using libraries like LangChain inside Spark transformations [1]:

- **Fixed-size chunking:** Splitting by an arbitrary character count (e.g., LangChain `CharacterTextSplitter`). Fast, but rarely works for production as it cuts off sentences.
- **Paragraph-based chunking:** Uses natural boundaries to preserve semantic coherence (e.g., LangChain `RecursiveCharacterTextSplitter`).
- **Format-specific chunking:** Splitting based on markdown headers or HTML tags to maintain structural context.
- **Semantic chunking:** Advanced techniques that analyze text to identify topic shifts and split accordingly (e.g., LangChain `SemanticChunker`).

When to Use vs. Alternatives

Use Fixed-size chunking only for rapid prototyping. For production RAG on Databricks, use Paragraph-based (RecursiveCharacter) or Format-specific chunking as the default to ensure chunks stand independently as meaningful units of text [1]. Use Semantic chunking when the source documents have no consistent format (like raw transcripts).

Common Mistakes

A common mistake is forgetting to include **overlap** between chunks. Databricks warns that failing to overlap chunks can cause important context to be lost at the split boundary, degrading retrieval results [1].

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
from pyspark.sql.functions import pandas_udf
import pandas as pd

@pandas_udf("array")
def chunk_text_udf(text_series: pd.Series) -> pd.Series:
    # Setup paragraph-based chunking strategy with overlap
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000,
        chunk_overlap=150
    )
    def chunk_doc(text):
        if text is None: return []
        return [doc.page_content for doc in splitter.create_documents([text])]

    return text_series.apply(chunk_doc)

# Apply chunking strategy to cleaned text
chunked_df = cleaned_df.withColumn("chunks", chunk_text_udf("parsed_text"))
```

Function Explanation: This code uses `RecursiveCharacterTextSplitter` from `LangChain` wrapped in a Databricks `pandas_udf`. It defines a chunk size (1000 characters) and a critical overlap (150 characters) to preserve continuity. The UDF returns an array of strings, converting one large document row into a list of manageable chunks.

Sources (Official Databricks Documentation)

- [1] Build an unstructured data pipeline for RAG: <https://docs.databricks.com/aws/en/agents/tutorials/ai-cookbook/quality-data-pipeline-rag>
- [2] `ai_parse_document` function: https://docs.databricks.com/aws/en/sql/language-manual/functions/ai_parse_document